

CS112.Q11.KHTN - Báo cáo giải pháp bài toán "Aura Tracker"

Nhóm 9

Ngày 19 tháng 11 năm 2025

1 Bài toán: Aura Tracker

Bài toán yêu cầu tính tổng lượng Aura (t_j) của m Hunter thuộc về vùng theo dõi i nhỏ nhất mà Hunter đó nằm **ng nghiêm ngặt bên trong** ($X_i < C_j < A_i$ và $Y_i < R_j < B_i$). Các vùng theo dõi không chồng lấn. Giới hạn: $W, H \leq 10^9$, $n, m \leq 10^5$.

2 Giải pháp theo Subtask

2.1 Subtask 1: $n, m \leq 10^3$

1. Phương pháp thiết kế thuật toán

Sử dụng ****Mô phỏng Trực tiếp (Direct Simulation / Brute Force)****.

2. Lý do phù hợp

Với số lượng vùng (n) và Hunter (m) nhỏ ($\leq 10^3$), mô phỏng trực tiếp cho phép kiểm tra điều kiện nghiêm ngặt cho mọi cặp (Hunter, Vùng) một cách đơn giản, dễ triển khai. Phương pháp này có độ phức tạp $O(n \cdot m) \leq 10^6$ phép toán, hoàn toàn đáp ứng được giới hạn thời gian (Time Limit).

3. Phân tích độ phức tạp

- **Độ phức tạp Thời gian ($O(T)$): $O(n \cdot m)$.**
 1. Thuật toán lặp qua m Hunter.
 2. Với mỗi Hunter, thuật toán lặp qua n vùng để kiểm tra điều kiện hình học.
 3. Tổng cộng $n \times m$ phép kiểm tra.
 - **Độ phức tạp Không gian ($O(S)$): $O(n + m)$.**
 1. Cần $O(n)$ để lưu trữ dữ liệu n vùng.
 2. Cần $O(m)$ để lưu trữ dữ liệu m Hunter.
-

2.2 Subtask 2: $W, H \leq 10^3$

1. Phương pháp thiết kế thuật toán

Sử dụng ****Thuật toán Dò quét Đường thẳng (Sweep-Line Algorithm) kết hợp Cây đoạn trên tọa độ trực tiếp****.

2. Lý do phù hợp

Số lượng n, m lớn ($\leq 10^5$) yêu cầu thuật toán $O(N \log N)$. Vì tọa độ trục Y nhỏ ($H \leq 10^3$), ta có thể:

- **Sweep-Line (X):** Chuyển bài toán 2D sang 1D, xử lý hiệu quả các sự kiện theo X .
- **Segment Tree (Không nén Y):** Xây dựng Cây đoạn trực tiếp trên dải $[0, H]$. Điều này giúp tránh chi phí nén tọa độ, giữ cho chi phí hằng số của thuật toán thấp.

3. Phân tích độ phức tạp

- **Độ phức tạp Thời gian ($O(T)$): $O((n + m) \log(n + m))$.**
 1. Bước sắp xếp $O(n + m)$ sự kiện mất $O((n + m) \log(n + m))$.
 2. Mỗi thao tác trên Cây đoạn xây dựng trên dải H tốn $O(\log H)$. Tổng chi phí xử lý sự kiện là $O((n + m) \log H)$.
 3. Do $H \leq 10^3$, $\log H$ là một hằng số rất nhỏ (< 10), nên độ phức tạp bị chi phối bởi bước sắp xếp.
 - **Độ phức tạp Không gian ($O(S)$): $O(n + m + H)$.**
 1. Lưu trữ input và sự kiện: $O(n + m)$.
 2. Cây đoạn được xây dựng trên dải $[0, H]$. Kích thước Cây đoạn là $O(H)$.
-

2.3 Subtask 3: Không ràng buộc ($n, m \leq 10^5, W, H \leq 10^9$)

1. Phương pháp thiết kế thuật toán

Sử dụng ****Thuật toán Dò quét Đường thẳng (Sweep-Line Algorithm)**** kết hợp ****Tọa độ nén**** và ****Cây đoạn Phi đệ quy (Iterative Segment Tree)****.

2. Lý do phù hợp

- **Tọa độ lớn (10^9):** Bắt buộc phải dùng **Nén tọa độ (Y)** để giảm dải tọa độ về $O(N)$ chỉ số, cho phép sử dụng Cây đoạn.
- **Hiệu suất tối ưu:** Sweep-Line kết hợp Segment Tree là cấu trúc tối ưu để xử lý **Range Assignment** (gán ID vùng) và **Point Query** (truy vấn Hunter) trong $O(\log N)$ thời gian.
- **Chống TLE:** Phiên bản **phi đệ quy** (Iterative) của Segment Tree được dùng để giảm thiểu chi phí gọi hàm Python (overhead), đảm bảo thuật toán đạt hiệu suất $O(N \log N)$ tối đa khi $N \approx 10^5$.

3. Phân tích độ phức tạp

Gọi K là tổng số tọa độ Y duy nhất ($K \leq 2n + m$).

- **Độ phức tạp Thời gian ($O(T)$): $O((n + m) \log(n + m))$.** Phân tích chi tiết:
 1. **Nén tọa độ Y :** Bước này bao gồm thu thập và sắp xếp K tọa độ. Chi phí sắp xếp: $O(K \log K) = O((n + m) \log(n + m))$.
 2. **Sắp xếp Sự kiện Sweep-Line:** Sắp xếp $2n + m$ sự kiện theo trục X mất $O((n + m) \log(n + m))$.
 3. **Xử lý Sự kiện:** Có $O(n + m)$ sự kiện. Mỗi sự kiện yêu cầu thao tác trên Cây đoạn, tốn $O(\log K)$ thời gian. Tổng cộng: $O((n + m) \log K) = O((n + m) \log(n + m))$.Độ phức tạp tổng thể bị chi phối bởi bước sắp xếp và xử lý sự kiện: **$O((n + m) \log(n + m))$.**
- **Độ phức tạp Không gian ($O(S)$): $O(n + m)$.** Phân tích chi tiết:
 1. **Lưu trữ Input/Events/Tọa độ Y :** Lưu trữ dữ liệu và danh sách $2n + m$ sự kiện. Chi phí: $O(n + m)$.
 2. **Cây đoạn (Segment Tree):** Cây đoạn được xây dựng trên K chỉ số đã nén. Kích thước của Cây đoạn phi đệ quy tỉ lệ tuyến tính với số lá, là $O(K)$. Chi phí: $O(K) = O(n + m)$.

Độ phức tạp không gian tổng thể là **$O(n + m)$** .

3 Mã nguồn Python Tối Ưu (Subtask 3)

Mã nguồn được tối ưu hóa cho tốc độ thực thi, sử dụng I/O nhanh và Segment Tree Phi đệ quy.

```
import sys
import math

def solve():
    input_data = sys.stdin.read().split()
    if not input_data:
        return

    W, H, n, m = int(input_data[0]), int(input_data[1]), int(input_data[2]), int(
        input_data[3])

    regions = []
    y_coords = set()
    data_index = 4
    for i in range(n):
        Xi = int(input_data[data_index])
        Yi = int(input_data[data_index + 1])
        Ai = int(input_data[data_index + 2])
        Bi = int(input_data[data_index + 3])
        regions.append((Xi, Yi, Ai, Bi, i))
        data_index += 4
        y_coords.add(Yi)
        y_coords.add(Bi)

    hunters = []
    for i in range(m):
        Cj = int(input_data[data_index])
        Rj = int(input_data[data_index + 1])
        tj = int(input_data[data_index + 2])
        hunters.append((Cj, Rj, tj))
        data_index += 3
        y_coords.add(Rj)

    y_sorted = sorted(list(y_coords))
    y_map = {y: i for i, y in enumerate(y_sorted)}

    N_Y = len(y_sorted)

    size = 1
    while size < N_Y:
        size <<= 1

    tree = [-1] * (2 * size)

    def update_iterative(L, R, val):
        L += size
        R += size

        while L <= R:
            if L & 1:
                tree[L] = val
                L += 1
            if not (R & 1):
                tree[R] = val
                R -= 1

            L >>= 1
            R >>= 1

    def query_iterative(idx):
        idx += size

        while idx >= 1:
            if tree[idx] != -1:
                return tree[idx]
            idx >>= 1
        return -1

    events = []
```

```

for Xi, Yi, Ai, Bi, region_id in regions:
    y_idx_start = y_map[Yi]
    y_idx_end = y_map[Bi]

    events.append((Xi, 0, y_idx_start, y_idx_end, region_id))
    events.append((Ai, 2, y_idx_start, y_idx_end, region_id))

for Cj, Rj, tj in hunters:
    Rj_idx = y_map[Rj]
    events.append((Cj, 1, Rj_idx, tj, -1))

events.sort(key=lambda x: (x[0], x[1]))

region_total_aura = [0] * n
untracked_aura = 0

for event in events:
    X, event_type, y_idx_start, y_idx_end, val = event

    l = y_idx_start + 1
    r = y_idx_end - 1

    if event_type == 0:
        if l <= r:
            update_iterative(l, r, val)

    elif event_type == 2:
        if l <= r:
            update_iterative(l, r, -1)

    elif event_type == 1:
        Rj_idx = y_idx_start
        aura = y_idx_end

        if 0 <= Rj_idx < N_Y:
            region_id = query_iterative(Rj_idx)
        else:
            region_id = -1

        if region_id != -1 and 0 <= region_id < n:
            region_total_aura[region_id] += aura
        else:
            untracked_aura += aura

sys.stdout.write('␣'.join(map(str, region_total_aura)) + '␣' + str(untracked_aura) +
                '\n')

solve()

```

Listing 1: aura_tracker_optimized.py - Giải pháp tối ưu cho Subtask 3